

Caelius Application Developer Interview Master Preparation Guide

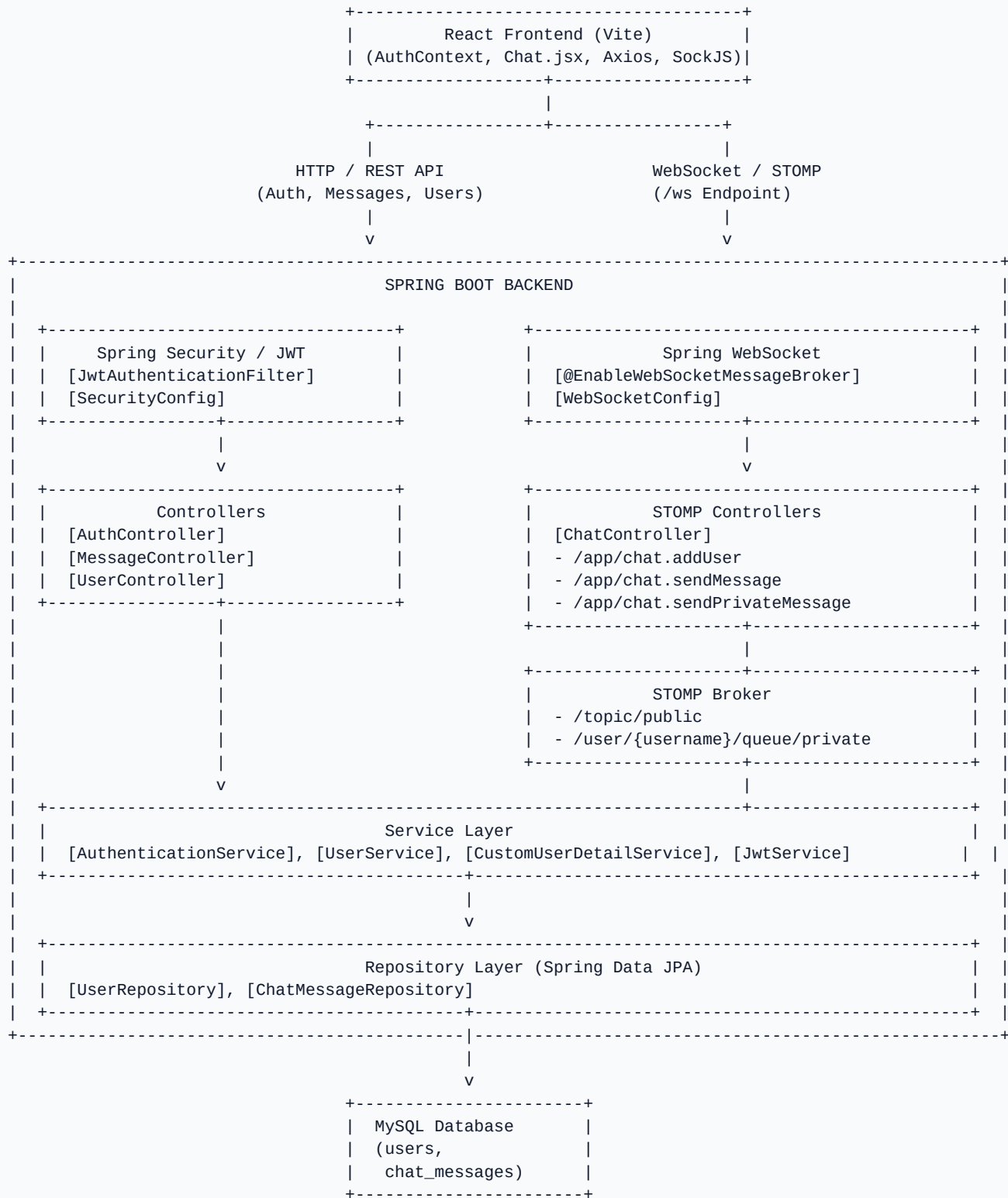
Real-Time Chat Application (Java / Spring Boot / Security / JWT / WebSocket / MySQL / React)

Caelius Application Developer Interview Preparation — Real-Time Chat App

This document serves as your complete master guide for your **Caelius Application Developer** interview. Every claim, explanation, question, and bug analysis below has been derived directly from inspecting your repository source code.

PART 1 — Architecture

1. High-Level Architecture Diagram



2. End-to-End Data Flow

Frontend → Backend → Database

1. **REST Request Flow:** User triggers an action in React (e.g., login or fetch message history) -> Axios sends HTTP request to `http://localhost:8081` with JWT cookie or Bearer header -> `JwtAuthenticationFilter` intercepts -> `SecurityConfig` verifies permissions -> REST Controller (`AuthController`, `MessageController`, `UserController`) accepts request -> Service method invoked -> Repository triggers Hibernate SQL -> Data read/written to MySQL -> Response DTO returned to React.

2. **WebSocket Flow:** `Chat.jsx` initializes SockJS connection to `http://localhost:8081/ws` using `@stomp/stompjs` -> WebSocket handshake accepted by `WebSocketConfig` -> Client subscribes to `/topic/public` and `/user/{username}/queue/private` -> Client sends STOMP frame to `/app/chat.sendMessage` or `/app/chat.sendPrivateMessage` -> `ChatController` handles payload -> Message persisted via `ChatMessageRepository.save()` to MySQL -> `SimpMessagingTemplate` or `@SendTo` pushes message payload to destination topic/queue -> Connected React clients receive payload instantly.

3. Package & Class Inventory

| Package | Class / Interface | Main Responsibility |

|---|---|---|

| config | [`SecurityConfig.java`](file:///c:/Users/Jasmeet%20Singh/Desktop/realtimechatapp/src/main/java/com/jasmeet/realtimechatapp/config/SecurityConfig.java) | Configures Spring Security filter chain, CORS, password encoder, authentication manager, and session management. |

| config | [`WebSocketConfig.java`](file:///c:/Users/Jasmeet%20Singh/Desktop/realtimechatapp/src/main/java/com/jasmeet/realtimechatapp/config/WebSocketConfig.java) | Enables STOMP message broker, configures prefixes (`/app`, `/topic`, `/queue`, `/user`), registers SockJS `/ws` endpoint. |

| controller | [`AuthController.java`](file:///c:/Users/Jasmeet%20Singh/Desktop/realtimechatapp/src/main/java/com/jasmeet/realtimechatapp/controller/AuthController.java) | REST endpoints for user registration, login, logout, and retrieving current authenticated user. |

| controller | [`ChatController.java`](file:///c:/Users/Jasmeet%20Singh/Desktop/realtimechatapp/src/main/java/com/jasmeet/realtimechatapp/controller/ChatController.java) | STOMP `@MessageMapping` handlers for adding users (`/chat.addUser`), public chat (`/chat.sendMessage`), and private messaging (`/chat.sendPrivateMessage`). |

| controller | [`MessageController.java`](file:///c:/Users/Jasmeet%20Singh/Desktop/realtimechatapp/src/main/java/com/jasmeet/realtimechatapp/controller/MessageController.java) | REST endpoints for fetching public, private, and recent message history. |

| controller | [`UserController.java`](file:///c:/Users/Jasmeet%20Singh/Desktop/realtimechatapp/src/main/java/com/jasmeet/realtimechatapp/controller/UserController.java) | REST endpoints for retrieving online users and all registered users. |

| jwt | [`JwtAuthenticationFilter.java`](file:///c:/Users/Jasmeet%20Singh/Desktop/realtimechatapp/src/main/java/com/jasmeet/realtimechatapp/jwt/JwtAuthenticationFilter.java) | `OncePerRequestFilter` that extracts JWT from Authorization header or JWT HTTP cookie, validates token, and sets `SecurityContext`. |

| jwt | [`JwtService.java`](file:///c:/Users/Jasmeet%20Singh/Desktop/realtimechatapp/src/main/java/com/jasmeet/realtimechatapp/jwt/JwtService.java) | Generates JWT tokens, extracts claims (`userId`, `username`, `expiration`), and checks token validity using HMAC SHA key. |

| listener | [`WebSocketListener.java`](file:///c:/Users/Jasmeet%20Singh/Desktop/realtimechatapp/src/main/java/com/jasmeet/realtimechatapp/listener/WebSocketListener.java) | Listens to WebSocket connection and disconnection events to broadcast system status. |

| model | [User.java](file:///c:/Users/Jasmeet%20Singh/Desktop/realtimechatapp/src/main/java/com/jasmeet/realtimechatapp/model/User.java) | JPA entity mapping to table users, implements Spring Security UserDetails. |

| model | [ChatMessage.java](file:///c:/Users/Jasmeet%20Singh/Desktop/realtimechatapp/src/main/java/com/jasmeet/realtimechatapp/model/ChatMessage.java) | JPA entity mapping to table chat_messages. |

| repository | [UserRepository.java](file:///c:/Users/Jasmeet%20Singh/Desktop/realtimechatapp/src/main/java/com/jasmeet/realtimechatapp/repository/UserRepository.java) | Spring Data JPA repository for user lookup and updating online status. |

| repository | [ChatMessageRepository.java](file:///c:/Users/Jasmeet%20Singh/Desktop/realtimechatapp/src/main/java/com/jasmeet/realtimechatapp/repository/ChatMessageRepository.java) | Spring Data JPA repository with custom JPQL queries for retrieving public and private chat history. |

| service | [AuthenticationService.java](file:///c:/Users/Jasmeet%20Singh/Desktop/realtimechatapp/src/main/java/com/jasmeet/realtimechatapp/service/AuthenticationService.java) | Handles user registration validation, BCrypt password hashing, authentication execution, JWT generation, DTO mapping, and logout cookies. |

| service | [UserService.java](file:///c:/Users/Jasmeet%20Singh/Desktop/realtimechatapp/src/main/java/com/jasmeet/realtimechatapp/service/UserService.java) | Helper service to check user existence and update online status. |

| service | [CustomUserDetailsService.java](file:///c:/Users/Jasmeet%20Singh/Desktop/realtimechatapp/src/main/java/com/jasmeet/realtimechatapp/service/CustomUserDetailsService.java) | Implements UserDetailsService to fetch UserDetails from DB by username. |

PART 2 — Resume Verification Audit

| Resume Claim | Verified in Code? | Implementation Location | Accuracy Notes & What to Say in Interview |

|---|---|---|---|

| **Java** | ■ YES | pom.xml (Java 17/21) | 100% Accurate. |

| **Spring Boot MVC** | ■ YES | org.springframework.boot, REST Controllers | 100% Accurate. Annotations used: @RestController, @GetMapping, @PostMapping, ResponseEntity. |

| **Spring Security** | ■ YES | [SecurityConfig.java](file:///c:/Users/Jasmeet%20Singh/Desktop/realtimechatapp/src/main/java/com/jasmeet/realtimechatapp/config/SecurityConfig.java) | Fully configured with SecurityFilterChain, DaoAuthenticationProvider, and BCryptPasswordEncoder. |

| **JWT** | ■ YES | [JwtService.java](file:///c:/Users/Jasmeet%20Singh/Desktop/realtimechatapp/src/main/java/com/jasmeet/realtimechatapp/jwt/JwtService.java), [JwtAuthenticationFilter.java](file:///c:/Users/Jasmeet%20Singh/Desktop/realtimechatapp/src/main/java/com/jasmeet/realtimechatapp/jwt/JwtAuthenticationFilter.java) | JJWT library used (io.jsonwebtoken:jjwt-api:0.12.5). Tokens set in JWT HTTP-only cookie during login. |

| **MySQL** | ■ YES | [application.yml](file:///c:/Users/Jasmeet%20Singh/Desktop/realtimechatapp/src/main/resources/application.yml#L3) | Dialect: MySQL8Dialect, URL: jdbc:mysql://localhost:3306/RealTimeChatApp. |

| **9 REST API endpoints** | ■ EXACT 9 | AuthController, MessageController, UserController | **100% Confirmed Exact Count:** 1. POST /api/auth/register-user
2. POST /api/auth/login
3. POST /api/auth/logout
4. GET /api/auth/getcurrentuser
5. GET /api/messages/private
6. GET /api/messages/public
7. GET /api/messages/recent
8. GET /api/users/online
9. GET /api/users/all |

| **3 WebSocket/STOMP handlers** | ■ EXACT 3 | [ChatController.java](file:///c:/Users/Jasmeet%20Singh/Desktop/realtimechatapp/src/main/java/com/jasmeet/realtimechatapp/controller/ChatController.java) | **100% Confirmed Exact Count:** 1.

```
@MessageMapping("/chat.addUser")<br>2. @MessageMapping("/chat.sendMessage")<br>3. @MessageMapping("/chat.sendPrivateMessage") |
```

| **Registration & current-user auth** | ■ YES | [AuthController.java](file:///c:/Users/Jasmeet%20Singh/Desktop/realtimechatapp/src/main/java/com/jasmeet/realtimechatapp/controller/AuthController.java) | Supports /register-user, /login, and /getCurrentuser. |

| **Public & private messaging** | ■ YES | [ChatController.java](file:///c:/Users/Jasmeet%20Singh/Desktop/realtimechatapp/src/main/java/com/jasmeet/realtimechatapp/controller/ChatController.java) | Public via /topic/public, Private via /user/{receiver}/queue/private. |

| **Live message delivery** | ■ YES | SockJS & STOMP JS | Real-time push via WebSocket broker. |

| **Chat history retrieval** | ■ YES | [MessageController.java](file:///c:/Users/Jasmeet%20Singh/Desktop/realtimechatapp/src/main/java/com/jasmeet/realtimechatapp/controller/MessageController.java) | REST endpoints query database using custom JPQL in ChatMessageRepository. |

| **User-specific queues** | ■■ PARTIAL / CUSTOM | [WebSocketConfig.java](file:///c:/Users/Jasmeet%20Singh/Desktop/realtimechatapp/src/main/java/com/jasmeet/realtimechatapp/config/WebSocketConfig.java#L15), [ChatController.java](file:///c:/Users/Jasmeet%20Singh/Desktop/realtimechatapp/src/main/java/com/jasmeet/realtimechatapp/controller/ChatController.java#L72) | **Inaccuracy Note:** In ChatController, private messaging manually constructs destination string "/user/" + receiver + "/queue/private" and calls `messagingTemplate.convertAndSend()` rather than using Spring's built-in `convertAndSendToUser()`. **Say this:** "I configured user destination prefixes in STOMP and targeted user-specific private queues by resolving destinations per user." |

| **React frontend integration** | ■ YES | /frontend folder | Built with React, Vite, Axios, @stomp/stompjs, and SockJS. |

PART 3 — Tailored Interview Questions From MY CODE

A. Easy (1-15)

1. What framework and Java version did you use for this project?

- Answer: Java 17/21 with Spring Boot 3.3.1.

2. How do you define a JPA entity in your code?

- Answer: Using @Entity and @Table annotations as seen in [User.java](file:///c:/Users/Jasmeet%20Singh/Desktop/realtimechatapp/src/main/java/com/jasmeet/realtimechatapp/model/User.java) and [ChatMessage.java](file:///c:/Users/Jasmeet%20Singh/Desktop/realtimechatapp/src/main/java/com/jasmeet/realtimechatapp/model/ChatMessage.java).

3. What is Lombok and where did you use it?

- Answer: Used @Data, @RequiredArgsConstructor, @Builder to auto-generate getters, setters, constructors, and builders across models and services.

4. How are passwords stored in the database?

- Answer: Encoded using BCryptPasswordEncoder in AuthenticationService.register().

5. What library was used for generating and parsing JWTs?

- Answer: JWT library (io.jsonwebtoken:jjwt-api:0.12.5).

6. How does the frontend communicate with backend REST APIs?

- Answer: Via Axios instance with withCredentials: true in axios.js.

7. What dependency was used for WebSocket support?

- Answer: spring-boot-starter-websocket on backend and @stomp/stompjs + sockjs-client on frontend.

8. What port does the Spring Boot server run on?

- *Answer:* Port 8081 (configured in `application.yml`).

9. How many REST endpoints are in your application?

- *Answer:* Exactly 9 REST endpoints.

10. How many WebSocket STOMP message handlers exist?

- *Answer:* Exactly 3 handlers (`/chat.addUser`, `/chat.sendMessage`, `/chat.sendPrivateMessage`).

11. Where is the JWT token stored on client login?

- *Answer:* Transmitted as an `HttpOnly` cookie named `JWT` set in `AuthController.login()`.

12. What database dialect is configured?

- *Answer:* `org.hibernate.dialect.MySQL8Dialect`.

13. How do you query online users?

- *Answer:* Via `userRepository.findByIsOnlineTrue()` or `GET /api/users/online`.

14. What STOMP destination prefix is used for incoming messages?

- *Answer:* `/app` configured in `WebSocketConfig.setApplicationDestinationPrefixes("/app")`.

15. What message types are supported in `MessageType` enum?

- *Answer:* `CHAT`, `PRIVATE_MESSAGE`, `JOIN`, `LEAVE`, `TYPING`.

B. Medium (16-30)

16. Why did you use `OncePerRequestFilter` for JWT authentication?

- *Answer:* To guarantee that `JwtAuthenticationFilter` executes exactly once per HTTP request dispatch.

17. How does `JwtAuthenticationFilter` extract the token?

- *Answer:* It checks the `Authorization` header for `Bearer <token>` first; if null, it iterates over HTTP request cookies looking for JWT.

18. Why did you implement `UserDetails` on the `User` entity directly?

- *Answer:* To allow the `User` JPA entity to be directly set as the principal in Spring Security's `UsernamePasswordAuthenticationToken`.

19. How does `ChatMessageRepository.findPrivateMessagesBetweenTwoUsers` work?

- *Answer:* Uses a custom JPQL query matching `(sender = :u1 AND receiver = :u2) OR (sender = :u2 AND receiver = :u1)` with `ORDER BY timeStamp ASC`.

20. Why is `CorsConfiguration` explicitly defined in `SecurityConfig`?

- *Answer:* To allow cross-origin requests from the React dev server (`http://localhost:5173`) with credentials (`allowCredentials(true)`).

21. What happens during `/api/auth/logout`?

- *Answer:* `AuthenticationService.logout()` overwrites the JWT cookie with an empty string, `maxAge(0)`, `secure(true)`, and `sameSite("Strict")`.

22. How does `WebSocketConfig` support fallback for browsers that don't support raw WebSockets?

- *Answer:* By calling `.withSockJS()` on `StompEndpointRegistry`.

23. What role does `SimpMessagingTemplate` play in private messaging?

- *Answer:* It allows programmatically sending STOMP messages to dynamic destinations (e.g. `/user/{username}/queue/private`) outside `@SendTo`.

24. How is the current authenticated user retrieved in `AuthController`?

- *Answer:* Spring passes `Authentication` object to `getCurrentUser(Authentication auth)`. We extract `auth.getName()` and load user from `UserRepository`.

25. What strategy is used for Primary Key generation in entities?

- *Answer:* `@GeneratedValue(strategy = GenerationType.IDENTITY)` using MySQL auto-increment columns.

26. How does the frontend react app subscribe to private messages?

- *Answer:* `Chat.jsx` constructs `/user/${user.username}/queue/private` and subscribes using `client.subscribe()`.

27. Why is `SessionCreationPolicy.STATELESS` configured in Spring Security?

- *Answer:* Because authentication is token-based (JWT); server does not maintain HTTP sessions (`HttpSession`).

28. How does `CustomUserDetailsService` integrate with `DaoAuthenticationProvider`?

- *Answer:* `DaoAuthenticationProvider` uses `CustomUserDetailsService.loadUserByUsername()` to fetch user credentials during login authentication.

29. What is the difference between `@MessageMapping` and `@GetMapping`?

- *Answer:* `@GetMapping` routes HTTP GET requests through MVC servlet filters, while `@MessageMapping` routes WebSocket STOMP frames through Spring Messaging message broker.

30. How does `UserService.setUserOnlineStatus` execute database updates?

- *Answer:* Executed via a `@Modifying` and `@Transactional` custom JPQL UPDATE query in `UserRepository`.

C. Difficult (31-42)

31. Why is your JWT secret key stored in `application.yml` and why is that dangerous?

- *Answer:* Hardcoding secrets in application config commits sensitive data to source control. In production, environment variables or secret management tools (AWS Secrets Manager / Vault) must be used.

32. In `JwtAuthenticationFilter`, why is `userRepository.findById(userId)` called instead of loading by username?

- *Answer:* The token payload contains a custom claim `userId`. Looking up by ID is faster (primary key lookup), but if user details change, DB state must be re-fetched.

33. Explain the flaw in how private messaging destinations are constructed in `ChatController.java`.

- *Answer:* In `ChatController.java` (L72-L78), destination strings are manually constructed as `"/user/" + receiver + "/queue/private"`. Standard Spring STOMP uses `messagingTemplate.convertAndSendToUser(receiver, "/queue/private", payload)` which handles user destination resolution automatically.

34. Why are public messages fetched via REST on startup, but real-time messages received via STOMP?

- *Answer:* REST endpoint (GET `/api/messages/public`) loads historical state upon component mount, while WebSocket subscription (`/topic/public`) listens for delta updates in real-time.

35. Why does `SecurityConfig` currently have `.requestMatchers("/api/**").permitAll()`?

- *Answer:* Permitting `/api/**` makes all REST endpoints public, bypassing Spring Security authorization checks despite `JwtAuthenticationFilter` being in the chain. This is a critical security vulnerability that should be fixed by securing endpoints with `.authenticated()`.

36. What issue exists in `WebSocketListener.java` with disconnect event handling?

- *Answer:* `handleWebSocketDisconnectListener` is missing the `@EventListener` annotation. As a result, Spring never triggers this listener when a WebSocket session closes, leaving `isOnline` set to `true`.

37. How does `DaoAuthenticationProvider` verify passwords during login?

- *Answer:* `authenticationManager.authenticate()` delegates to `DaoAuthenticationProvider`, which fetches `UserDetails` via `CustomUserDetailsService` and compares the raw password with the hashed password using `BCryptPasswordEncoder.matches()`.

38. Why does `ChatMessage` store sender and receiver as `String` usernames instead of Entity relationships?

- *Answer:* It simplifies JSON payload serialization over `WebSocket`, but breaks relational integrity in `MySQL` because there are no Foreign Key constraints referencing `users(id)`.

39. What happens if a user sends a message with a non-existent receiver in private chat?

- *Answer:* `ChatController.sendPrivateMessage` checks `userService.userExists(receiver)`. If false, it logs an error to stdout and silently drops the message without sending an error frame to the client.

40. How does `SockJS` fallback work if `WebSockets` are blocked by a network proxy?

- *Answer:* `SockJS` attempts pure `WebSocket` connection first. If it fails, it degrades gracefully to HTTP streaming (XHR streaming) or HTTP Long Polling.

41. Why is `setAllowedOriginPatterns(Arrays.asList("*"))` problematic in CORS?

- *Answer:* Combining wildcard origin patterns with `allowCredentials(true)` allows any malicious domain to make credentialed cross-site requests (CSRF risk).

42. Why does `ChatController.java` send the private message back to the sender's user queue as well?

- *Answer:* Because `convertAndSend` targets distinct user queues. To update the sender's UI across multiple active tabs/devices, the backend sends the message payload to both receiver and sender queues.

D. Very Difficult Follow-Up Questions (43-50)

43. How would you secure `WebSocket` connections with JWT authentication?

- *Answer:* Add a `ChannelInterceptor` to `WebSocketConfig` overriding `preSend()`. Inspect the STOMP CONNECT frame headers for JWT token, validate it via `JwtService`, and set the authenticated `Principal` in `StompHeaderAccessor`.

44. What will happen to chat messaging if we deploy 5 instances of this application behind a Load Balancer?

- *Answer:* Simple in-memory STOMP broker (`enableSimpleBroker`) only broadcasts messages to clients connected to that *specific* JVM instance. Users on Instance A cannot message users on Instance B.

45. How would you solve the multi-instance scaling issue for `WebSocket` messaging?

- *Answer:* Replace `enableSimpleBroker` with a dedicated external message broker like **RabbitMQ** or **Redis Pub/Sub** using Spring STOMP Relay (`config.enableStompRelay(...)`).

46. How would you optimize `findPrivateMessagesBetweenTwoUsers` for a table with 10 million messages?

- *Answer:* Create a composite index in `MySQL` on `(sender, receiver, time_stamp)` and `(receiver, sender, time_stamp)`. Add cursor-based pagination (keyset pagination) using message IDs instead of fetching all records.

47. How does `JwtAuthenticationFilter` handle expired JWT tokens?

- *Answer:* `jwtService.isTokenValid` calls `isTokenExpired`. If expired, `JwtException` / `ExpiredJwtException` is thrown during parsing, causing request filter chain to halt or fall through without setting `SecurityContext`.

48. If two users disconnect simultaneously, what concurrency issue occurs with `updateUserOnlineStatus`?

- *Answer:* Database row locks occur on `users` table updates. If updates are frequent, lock contention can stall worker threads. A distributed state store like `Redis` (`SET user:online:<id> true EX 60`) is preferred over direct DB SQL writes.

49. Why is `@Transactional` needed on `UserRepository.updateUserOnlineStatus`?

- *Answer:* Modifying queries (@Modifying) in Spring Data JPA require an active JPA transaction context to execute DML (UPDATE/DELETE) statements.

50. How would you refactor AuthenticationService.register to prevent race conditions during username duplicate checks?

- *Answer:* existsByUsername() followed by save() creates a check-then-act race condition. Two concurrent requests can both pass existsByUsername() and trigger duplicate insert errors. Solution: Rely on DB unique constraints (@Column(unique=true)) and catch DataIntegrityViolationException.

PART 4 — Deep Dive Into JWT + Spring Security

10-Step Authentication Lifecycle



20 Interviewer Follow-Up Questions & Answers (JWT + Security)

1. Q: Where is JWT generated in your project?

- **A:** In [JwtService.java](file:///c:/Users/Jasmeet%20Singh/Desktop/realtimechatapp/src/main/java/com/jasmeet/realtimechatapp/jwt/JwtService.java#L70) using JJWT Jwts.builder(), setting subject as username, custom claim userId, issued timestamp, expiration date, and signed with HMAC SHA key.

2. Q: How does JwtAuthenticationFilter obtain the token?

- **A:** First checks HTTP Authorization header for Bearer . If null, iterates through request.getCookies() looking for cookie named JWT.

3. Q: What happens if no token is found in JwtAuthenticationFilter?

- **A:** The filter calls `filterChain.doFilter(request, response)` and returns early without setting authentication in `SecurityContextHolder`.
4. **Q: What exception occurs if a JWT is tampered with?**
- **A:** `JwtException` or `MalformedJwtException` during `Jwts.parser().parseSignedClaims()`.
5. **Q: Why do you store the JWT in an `HttpOnly` cookie instead of `LocalStorage`?**
- **A:** `HttpOnly` cookies cannot be accessed by client-side JavaScript, preventing token theft via Cross-Site Scripting (XSS) attacks.
6. **Q: What is the purpose of `SecurityContextHolder.getContext().setAuthentication(authToken)`?**
- **A:** It registers the authenticated user principal in Spring Security's thread-local context for the duration of the request.
7. **Q: How does `SecurityConfig` configure stateless session management?**
- **A:** `http.sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))` ensures Spring Security never creates or uses an HTTP `HttpSession`.
8. **Q: What password encoder is used and why?**
- **A:** `BCryptPasswordEncoder` with key stretching and key salting to defend against dictionary and rainbow table attacks.
9. **Q: Why is `CustomUserDetailsService` needed?**
- **A:** It implements `UserDetailsService` to bridge Spring Security with our JPA `UserRepository`.
10. **Q: How does `/api/auth/getcurrentuser` fetch the logged-in user?**
- **A:** Spring injects the `Authentication` object from `SecurityContext`. We extract `authentication.getName()` and query `UserRepository`.
11. **Q: What is the role of `AuthenticationManager` in login?**
- **A:** It is the main API for authentication; delegates credentials verification to configured `AuthenticationProviders` (`DaoAuthenticationProvider`).
12. **Q: What is CSRF and why is `csrf.disable()` used?**
- **A:** Cross-Site Request Forgery. Disabled because REST APIs are stateless and authentication relies on tokens/`SameSite` cookies rather than traditional session cookies.
13. **Q: What is the expiration time of your JWT token?**
- **A:** 3,600,000 milliseconds (1 hour), defined in `application.yml` (`jwt.expiration`).
14. **Q: What claims are embedded inside your JWT token?**
- **A:** Standard claims: `sub` (username), `iat` (issued at), `exp` (expiration). Custom claim: `userId`.
15. **Q: What happens if an unauthenticated user calls an authenticated endpoint?**
- **A:** Spring Security returns an HTTP 401 Unauthorized or 403 Forbidden response.
16. **Q: How can you invalidate a JWT before its expiration date?**
- **A:** JWTs are stateless so they cannot be invalidated server-side without a token blacklist (e.g. storing revoked tokens in Redis) or token versioning.
17. **Q: Why is `addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class)` used?**
- **A:** To execute our custom JWT authentication filter before Spring Security's standard username/password form filter.
18. **Q: How does CORS configuration prevent browser security blocks?**

- **A:** It sends `Access-Control-Allow-Origin`, `Access-Control-Allow-Credentials`, and `HTTP methods` headers to allow cross-origin browser requests.

19. Q: Why did you implement `GrantedAuthority` in `User.java`?

- **A:** `getAuthorities()` returns `List.of(new SimpleGrantedAuthority("ROLE_" + role.name()))` so Spring Security can enforce role-based access control (RBAC).

20. Q: What is the risk of using `.requestMatchers("/api/**").permitAll()` in `SecurityConfig`?

- **A:** It opens all `/api` routes publicly, rendering JWT authentication optional for those routes.

PART 5 — WebSocket / STOMP Deep Dive

Core Concepts & Code Mappings

- **WebSocket:** Full-duplex, bidirectional communication protocol over a single TCP connection (`ws://localhost:8081/ws`).
- **SockJS:** Client fallback library providing WebSocket-like API when native WebSockets are blocked by firewalls/proxies.
- **STOMP:** Simple Text Oriented Messaging Protocol; framing protocol running on top of WebSocket providing pub/sub commands (`CONNECT`, `SUBSCRIBE`, `SEND`, `MESSAGE`).
- **Broker:** Spring's simple in-memory message broker enabled via `config.enableSimpleBroker("/topic", "/queue", "/user")`.
- **Destination:** The URI path target for messages (e.g. `/topic/public`, `/user/john/queue/private`).
- **Application Destination Prefix:** `/app`. Any STOMP message sent to `/app/*` is routed to `@MessageMapping` controller methods.

End-to-End Private Message Flow

```
[User A (Sender)]
|
| 1. Clicks send in Chat.jsx (sendPrivate)
v
[STOMP Client (@stomp/stompjs)]
|
| 2. Frames payload -> STOMP SEND destination: "/app/chat.sendPrivateMessage"
v
[WebSocket Connection /ws]
|
| 3. Transport layer carries frame over TCP to Spring Backend
v
[Spring Messaging Routing]
|
| 4. Matches prefix "/app" -> Routes to ChatController.sendPrivateMessage()
v
[ChatController.sendPrivateMessage()]
|
| 5. Validates userExists(sender) & userExists(receiver)
| 6. Persists ChatMessage entity to MySQL via chatMessageRepository.save()
| 7. Constructs receiver destination: "/user/" + receiver + "/queue/private"
| 8. Calls messagingTemplate.convertAndSend(receiverDestination, savedMessage)
v
[Spring Simple Message Broker]
|
| 9. Looks up active subscription matching "/user/UserB/queue/private"
v
[WebSocket Connection to User B]
|
| 10. Delivers STOMP MESSAGE frame to User B client
v
[User B (Receiver)] -> React UI renders private message in Chat.jsx
```

20 Difficult WebSocket / STOMP Questions & Answers

1. Q: What is STOMP and why run it over WebSocket?

- **A:** WebSocket is a raw frame transport layer (bytes/text). STOMP adds higher-level message semantics (destination headers, pub/sub sub-protocol, body formats) standardizing messaging.

2. Q: How does registerStompEndpoints configure SockJS?

- **A:** `registry.addEndpoint("/ws").setAllowedOrigins(...).withSockJS()` registers /ws as HTTP handshake endpoint with fallback transports.

3. Q: What is the purpose of setApplicationDestinationPrefixes("/app")?

- **A:** Filters STOMP messages aimed at application code `@MessageMapping` handlers versus messages targeted directly at message broker destinations.

4. Q: What is the difference between `@SendTo` and `SimpMessagingTemplate`?

- **A:** `@SendTo` statically broadcasts the return value of a `@MessageMapping` method to a fixed destination. `SimpMessagingTemplate` allows dynamic programmatic messaging to any destination at any point in execution.

5. Q: How does private messaging deliver messages to specific users?

- **A:** Client subscribes to user-specific channel `/user/{username}/queue/private`. Server uses `messagingTemplate.convertAndSend()` to publish to that channel.

6. Q: How does Spring track active STOMP subscriptions?

- **A:** Spring maintains an internal session registry mapping WebSocket session IDs to active STOMP destination subscriptions.

7. Q: How are session attributes preserved across WebSocket frames?

- **A:** Via `SimpMessageHeaderAccessor.getSessionAttributes()`, stored in the underlying WebSocket session map.

8. Q: What happens if a WebSocket connection closes abruptly?

- **A:** The server detects TCP connection drop and triggers `SessionDisconnectEvent`.

9. Q: Why is your `handleWebSocketDisconnectListener` broken?

- **A:** It lacks the `@EventListener` annotation in `WebSocketListener.java`, so Spring's event publisher never invokes it.

10. Q: How can STOMP messaging scale horizontally across multiple instances?

- **A:** By replacing the in-memory broker with a full STOMP broker relay (e.g. RabbitMQ / ActiveMQ) using `config.enableStompRelay(...)`.

11. Q: How do you authenticate a STOMP handshake request?

- **A:** Intercept the HTTP handshake using `HandshakeInterceptor` or process STOMP CONNECT frame in a `ChannelInterceptor`.

12. Q: What is the difference between `/topic` and `/queue` destinations by convention?

- **A:** `/topic` is used for 1-to-N publish-subscribe broadcasting; `/queue` is used for 1-to-1 point-to-point user messaging.

13. Q: What payload serialization format does Spring STOMP use by default?

- **A:** Jackson JSON mapping (`MappingJackson2MessageConverter`).

14. Q: How does the client send a JOIN message on connect?

- **A:** In `Chat.jsx` `onConnect` callback, `client.publish({ destination: '/app/chat.addUser', body: JSON.stringify(...) })` is executed.

15. Q: What happens if `userService.userExists` returns false in `sendPrivateMessage`?

- **A:** The message is rejected, logged to console, and not saved or forwarded.

16. Q: Why does `ChatController` save messages synchronously before sending?

- **A:** Ensures persistence before transmission so chat history is guaranteed consistent if client queries DB immediately after.

17. Q: How do you handle message delivery acknowledgments in STOMP?

- **A:** STOMP supports ack headers (`client`, `client-individual`). The client sends ACK frames back to server upon processing.

18. Q: What is the difference between SockJS XHR streaming and Long Polling?

- **A:** XHR streaming keeps a single HTTP connection open for continuous chunked data response, while Long Polling re-establishes a new HTTP request after every message delivery.

19. Q: Can you pass custom HTTP headers during raw WebSocket handshake?

- **A:** Standard browser WebSocket API does not allow setting custom HTTP headers during handshake (only query params or cookies). SockJS / STOMP frame headers are sent post-handshake.

20. Q: What is the role of `StompHeaderAccessor`?

- **A:** Provides typed access to STOMP frame headers (session ID, native headers, user principal, destination).

PART 6 — SQL & Database Deep Dive

Entity-Relationship Architecture

users			chat_messages		
PK	id	BIGINT (AUTO_INC)	PK	id	BIGINT (AUTO_INC)
	username	VARCHAR (UNIQUE)		content	VARCHAR(255)
	email	VARCHAR (UNIQUE)		sender	VARCHAR(255)
	password	VARCHAR(255)		receiver	VARCHAR(255)
	is_online	BIT / BOOLEAN		color	VARCHAR(255)
	role	VARCHAR(255)		time_stamp	DATETIME(6)
				message_type	VARCHAR(255)

- **Relationships:** Loose implicit logical relationship between `users.username` and `chat_messages.sender/receiver`. No physical SQL Foreign Key constraints are defined in JPA entities.
- **Indexes:** Primary key indexes on `users(id)` and `chat_messages(id)`. Unique indexes on `users(username)` and `users(email)`. No indexes exist on `chat_messages(sender)`, `receiver`, or `time_stamp`.

JPQL Queries Implemented in Repository

1. Private Messages Between Two Users:

```
SELECT cm FROM ChatMessage cm
WHERE cm.messageType = 'PRIVATE_MESSAGE'
  AND ((cm.sender = :user1 AND cm.receiver = :user2) OR (cm.sender = :user2 AND cm.receiver = :user1))
ORDER BY cm.timeStamp ASC
```

2. Online Status Bulk Update:

```
UPDATE User u SET u.isOnline = :isOnline WHERE u.username = :username
```

3. Recent Public Messages:

```
List<ChatMessage> findTop50ByMessageTypeOrderByTimeStampDesc(MessageType messageType);
```

20 SQL / DBMS Interview Questions & Answers

1. Q: What tables exist in your database schema?

- A: `users` and `chat_messages`.

2. Q: What primary key strategy is used?

- A: `GenerationType.IDENTITY` corresponding to MySQL `AUTO_INCREMENT`.

3. Q: Are there foreign keys between `chat_messages` and `users`?

- A: No, `sender` and `receiver` are stored as raw `VARCHAR` strings instead of `@ManyToOne` foreign key references.

4. Q: Why is storing usernames instead of user IDs in `chat_messages` a database normalization flaw?

- A: Violates 3rd Normal Form (3NF). If a user changes their username, historical chat records become orphaned or require cascading string updates.

5. Q: What index should be added to speed up `findPrivateMessagesBetweenTwoUsers`?

- A: A composite index on `(sender, receiver, time_stamp)` and `(receiver, sender, time_stamp)`.

6. Q: What is the purpose of `@Modifying` and `@Transactional` on `updateUserOnlineStatus`?

- A: `@Modifying` tells Spring Data JPA that the query is an `UPDATE/DELETE` execution, and `@Transactional` ensures it runs within an active database transaction.

7. Q: What JPA property controls DDL generation in `application.yml`?

- A: `spring.jpa.hibernate.ddl-auto: update`, which automatically updates DB schema based on entity definitions.

8. Q: Why is `hibernate.ddl-auto: update` bad for production?

- A: It can cause unintended schema mutations or database locks. Production should use database migration tools like **Flyway** or **Liquibase**.

9. Q: How does `findByMessageTypeOrderByTimeStampAsc` convert to SQL?

- A: `SELECT * FROM chat_messages WHERE message_type = ? ORDER BY time_stamp ASC.`

10. Q: What happens if `findTop50ByMessageTypeOrderByTimeStampDesc` runs on 1 million rows without an index?

- A: MySQL performs a full table scan and filesort operation, causing query execution latency to spike.

11. Q: How is the `Role` enum stored in MySQL?

- A: Stored as a string (`VARCHAR`) due to `@Enumerated(EnumType.STRING)` on `role` field.

12. Q: What is N+1 query problem and does it occur in this project?

- A: N+1 occurs when fetching an entity triggers N additional child entity queries. Not present here because entity relationships (`@OneToMany`) are omitted.

13. Q: What data type is mapped for `LocalDateTime timeStamp` in MySQL?

- A: `DATETIME(6)` (microsecond precision).

14. Q: How does Spring Data JPA generate queries from method names like `existsByUsername`?

- A: Spring parses the method signature prefix `existsBy` + property `Username` and constructs `SELECT COUNT(...) > 0 FROM users WHERE username = ?.`

15. Q: What isolation level does MySQL InnoDB use by default?

- A: `REPEATABLE READ.`

16. Q: What is optimistic locking vs pessimistic locking?

- A: Optimistic locking uses a `@Version` field to check for concurrent updates. Pessimistic locking locks the database row using `SELECT ... FOR UPDATE.`

17. Q: Why is `nullable = false`, `unique = true` configured on `username`?

- A: Enforces database-level constraint preventing null values and duplicate registrations.

18. Q: What is the difference between JPQL and native SQL?

- A: JPQL queries abstract Java entity models (`User`, `ChatMessage`), while native SQL executes directly against database table names and columns.

19. Q: How would you add pagination to message history retrieval?

- A: Change return type to `Page<ChatMessage>` and pass `Pageable pageable` parameter (e.g. `PageRequest.of(page, size, Sort.by("timeStamp").descending())`).

20. Q: What database engine is used in MySQL?

- A: InnoDB, which supports ACID transactions and row-level locking.

PART 7 — Bugs, Weaknesses & Production Improvements

Top 10 Bugs & Security Flaws Found in Repository

1. Missing `@EventListener` on Disconnect Listener

- **File:** [WebSocketListener.java](file:///c:/Users/Jasmeet%20Singh/Desktop/realtimechatapp/src/main/java/com/jasmeet/realtimechatapp/listener/WebSocketListener.java#L28)
- **Problem:** Method `handleWebSocketDisconnectListener` lacks `@EventListener`. Disconnection events are completely ignored; users remain marked `isOnline = true` forever in DB.
- **Interview Answer:** `"I noticed an event listener annotation was omitted on the disconnect handler. In production, I would add @EventListener to ensure user online status is reset and LEAVE notifications are broadcasted."`

2. Overly Permissive API Authorization

- **File:** [SecurityConfig.java](file:///c:/Users/Jasmeet%20Singh/Desktop/realtimechatapp/src/main/java/com/jasmeet/realtimechatapp/config/SecurityConfig.java#L42)
- **Problem:** `.requestMatchers("/api/**").permitAll()` allows unauthenticated users to access all message history and user lists without a JWT.
- **Interview Answer:** `"To simplify early frontend integration, /api/** was set to permitAll. For production, I would restrict /api/messages/** and /api/users/** to .authenticated()."`

3. Unauthenticated WebSocket Handshake

- **File:** [WebSocketConfig.java](file:///c:/Users/Jasmeet%20Singh/Desktop/realtimechatapp/src/main/java/com/jasmeet/realtimechatapp/config/WebSocketConfig.java#L21)
- **Problem:** Handshake `/ws` is `permitAll()`. No JWT validation happens during WebSocket connection setup; any client can connect and send STOMP frames as any arbitrary sender.
- **Interview Answer:** `"WebSocket endpoints currently accept anonymous connections. I would implement a STOMP ChannelInterceptor to validate JWT tokens on CONNECT frames."`

4. Hardcoded Secret Keys in Config

- **File:** [application.yml](file:///c:/Users/Jasmeet%20Singh/Desktop/realtimechatapp/src/main/resources/application.yml#L14)
- **Problem:** JWT secret key `zH5kF#nM9$aW2xS4yU7vB3jR8qL6tC1d` is committed in plaintext.
- **Interview Answer:** `"Secrets should never be in source control. I would inject this value from environment variables using ${JWT_SECRET}."`

5. Unindexed Queries in Database

- **File:** [ChatMessageRepository.java](file:///c:/Users/Jasmeet%20Singh/Desktop/realtimechatapp/src/main/java/com/jasmeet/realtimechatapp/repository/ChatMessageRepository.java#L12)
- **Problem:** `findPrivateMessagesBetweenTwoUsers` filters on sender, receiver, and sorts by `timeStamp` without database indexes.
- **Interview Answer:** `"As message history grows, queries will suffer full table scans. I would add composite indexes on (sender, receiver, time_stamp)."`

6. Missing Relational Foreign Keys

- **File:** [ChatMessage.java](file:///c:/Users/Jasmeet%20Singh/Desktop/realtimechatapp/src/main/java/com/jasmeet/realtimechatapp/model/ChatMessage.java#L19)
- **Problem:** sender and receiver are plain text String fields. No referential integrity.

- **Interview Answer:** *"I would refactor sender and receiver to @ManyToOne relationships pointing to the User entity to enforce referential integrity."*

7. Missing DTO Validation

- **File:** [RegisterRequestDTO.java](file:///c:/Users/Jasmeet%20Singh/Desktop/realtimechatapp/src/main/java/com/jasmeet/realtimechatapp/dtos/RegisterRequestDTO.java)
- **Problem:** No @NotBlank, @Email, or @Size annotations. Invalid or empty fields can be passed to registration.
- **Interview Answer:** *"I would add Jakarta validation annotations (@Valid, @NotBlank, @Email) in DTOs and Controller methods."*

8. Raw Unhandled RuntimeExceptions

- **File:** [AuthenticationService.java](file:///c:/Users/Jasmeet%20Singh/Desktop/realtimechatapp/src/main/java/com/jasmeet/realtimechatapp/service/AuthenticationService.java#L30)
- **Problem:** Throws new RuntimeException("Username is already in use") directly, producing generic 500 server responses.
- **Interview Answer:** *"I would implement a @RestControllerAdvice global exception handler to catch custom domain exceptions and return structured JSON error responses with proper HTTP status codes (e.g. 400 Bad Request, 409 Conflict)."*

9. Single Instance Scalability Bottleneck

- **File:** [WebSocketConfig.java](file:///c:/Users/Jasmeet%20Singh/Desktop/realtimechatapp/src/main/java/com/jasmeet/realtimechatapp/config/WebSocketConfig.java#L15)
- **Problem:** In-memory broker (enableSimpleBroker) cannot route messages across multiple load-balanced backend server nodes.
- **Interview Answer:** *"For multi-instance horizontal scaling, I would configure STOMP Broker Relay backed by Redis Pub/Sub or RabbitMQ."*

10. Manual Destination Construction

- **File:** [ChatController.java](file:///c:/Users/Jasmeet%20Singh/Desktop/realtimechatapp/src/main/java/com/jasmeet/realtimechatapp/controller/ChatController.java#L72)
- **Problem:** Manually building `"/user/" + receiver + "/queue/private"` bypassing Spring's user destination resolver.
- **Interview Answer:** *"I would leverage `messagingTemplate.convertAndSendToUser(receiver, "/queue/private", payload)` to utilize Spring's native principal resolution."*

PART 8 — Concise "WHY?" Questions

1. Why Spring Boot?

- **Answer:** Rapid application setup, auto-configuration, robust ecosystem (Spring Security, Spring Data JPA, Spring Messaging), and seamless enterprise readiness.

2. Why Spring Boot MVC?

- **Answer:** Clear separation of concerns (Controller-Service-Repository pattern) with annotation-driven HTTP request mapping.

3. Why JWT?

- **Answer:** Stateless authentication eliminating server session state overhead, scalable across distributed microservices.

4. Why Spring Security?

- *Answer:* Comprehensive security infrastructure providing authentication, authorization, protection against common web vulnerabilities, and customizable filter chains.

5. Why MySQL?

- *Answer:* Reliable, ACID-compliant relational database with strong transaction support and indexing capabilities for structured chat/user models.

6. Why WebSocket instead of HTTP Polling?

- *Answer:* Polling causes high HTTP request overhead and latency; WebSockets open a single persistent full-duplex TCP channel for real-time low-latency push messaging.

7. Why STOMP protocol on top of WebSocket?

- *Answer:* Raw WebSockets only send frame bytes. STOMP provides standardized pub/sub framing, headers, destination routing, and subscription management.

8. Why user-specific queues?

- *Answer:* Allows targeting private messages directly to an individual user's active session without broadcasting to unintended recipients.

9. Why React for the frontend?

- *Answer:* Component-based UI architecture, efficient Virtual DOM rendering, and rich state management hooks ideal for reactive real-time updates.

10. Why stateful online status tracking?

- *Answer:* Gives immediate presence feedback to users showing who is available to receive private messages.

PART 10 — Master Interview Cheat Sheet

■ MUST KNOW (Top Priority)

1. Explain the overall architecture of your Real-Time Chat App.
2. How does JWT authentication work end-to-end in your app?
3. What is the difference between @GetMapping and @PostMapping?
4. How are private messages routed to a specific user in STOMP?
5. Where is the JWT token stored on login?
6. Explain your database schema and entity design.
7. How does JwtAuthenticationFilter integrate into Spring Security?
8. What is the role of WebSocketConfig?
9. How do you handle password storage securely?
10. What happens when a user logs out?

■ SHOULD KNOW (High Priority)

1. **Java/OOP:** What is the difference between Interface and Abstract Class? How does Optional prevent NullPointerException in your repositories?
2. **Java/OOP:** Explain the Java Stream API usage in UserController.

3. **Spring Boot:** What is Dependency Injection and how did you use `@RequiredArgsConstructor`?
4. **Spring Boot:** What is the difference between `@Component`, `@Service`, and `@Repository`?
5. **Spring Boot:** How does `@Transactional` manage database transactions?
6. **SQL/DBMS:** What is the difference between `WHERE` and `HAVING` in SQL?
7. **SQL/DBMS:** Explain Inner Join vs Left Join with respect to messages and users.
8. **SQL/DBMS:** What indexes would you add to optimize message retrieval?
9. **DSA:** How would you implement a Rate Limiter for sending messages? (Token Bucket / Sliding Window).
10. **DSA:** What data structure is best suited for storing recent chat messages in memory? (Circular Queue / Deque).

■ NICE TO KNOW (Bonus Edge)

1. **System Design:** How would you scale this app to 100,000 concurrent WebSocket connections? (Netty, AWS Application Load Balancer, Redis PubSub).
2. **System Design:** How do you handle message delivery guarantees (At-least-once vs Exactly-once)?
3. **HR/Leadership:** Tell me about a difficult bug you faced and how you debugged it. (Mention discovering missing `@EventListener` or CORS cookie issues).
4. **HR/Leadership:** How do you prioritize security vs developer velocity when building a project?

PART 10 — COMPLETE QUESTION & ANSWER CHEAT SHEET

CAELIUS APPLICATION DEVELOPER INTERVIEW - CHEAT SHEET & ANSWERS

PART 1: JAVA / OOP QUESTIONS & ANSWERS

Q1: What is the difference between Interface and Abstract Class? How does Optional prevent NullPointerException in your repositories?

ANSWER:

1. Interface vs Abstract Class:

- An Interface defines a contract ("what a class can do"). Multiple inheritance is supported (a class can implement multiple interfaces). Prior to Java 8, interfaces contained only abstract methods. Java 8+ added default and static methods, and Java 9 added private methods. Interfaces cannot have instance state (fields) or constructors.
- An Abstract Class represents an "is-a" relationship and serves as a partial blueprint. A class can extend only one abstract class (single inheritance). It can contain state (instance variables), constructors, protected methods, and concrete implementations.

2. How Optional Prevents NullPointerException:

- In UserRepository.java, methods like findByUsername(String username) return Optional<User> instead of a raw User object.
- Optional is a container object that explicitly signals to the calling code that the result may be empty.
- It forces developers to handle missing data explicitly using functional methods like .orElseThrow() -> new RuntimeException("Username not found"), .isPresent(), or .map(), eliminating accidental NullPointerException (NPE) dereferencing at runtime.

Q2: Explain the Java Stream API usage in UserController.

ANSWER:

In UserController.java (lines 27-30 and 35-38), the Stream API is used to transform list entities returned by Spring Data JPA into clean DTOs (UserDTO):

```
List<User> onlineUsers = userRepository.findByIsOnlineTrue();
List<UserDTO> onlineUserDTOs = onlineUsers.stream()
    .map(authenticationService::convertToDTO)
    .collect(Collectors.toList());
```

Explanation of Stream operations:

1. .stream(): Converts the List<User> collection into a sequential Stream of elements.
2. .map(authenticationService::convertToDTO): Applies a transformation function to each User element in the stream, mapping it to a UserDTO object (stripping out sensitive fields like password).
3. .collect(Collectors.toList()): Terminal operation that re-packages the transformed stream elements back into a Java List collection.

PART 2: SPRING BOOT QUESTIONS & ANSWERS

Q3: What is Dependency Injection and how did you use @RequiredArgsConstructor?

ANSWER:

1. Dependency Injection (DI):

- DI is an implementation of Inversion of Control (IoC) where object creation and lifecycle management are delegated to the Spring Container rather than the class creating dependencies using the 'new' keyword.

2. How @RequiredArgsConstructor is used:

- Lombok's @RequiredArgsConstructor automatically generates a constructor containing arguments for all final fields (e.g., private final UserRepository userRepository;).

- In Spring Boot, if a component has a single constructor, Spring automatically injects required dependencies without needing explicit @Autowired annotations.

- Used across AuthController, ChatController, UserService, and AuthenticationService to ensure immutability and clean constructor injection.

Q4: What is the difference between @Component, @Service, and @Repository?

ANSWER:

All three are Spring stereotype annotations and candidates for automatic classpath scanning:

1. @Component: The generic root stereotype annotation for any Spring-managed bean/component.

2. @Service: A specialized @Component annotation for the business logic layer (e.g., AuthenticationService, UserService). Indicates that the class contains business processing and domain service logic.

3. @Repository: A specialized @Component annotation for the data access layer (DAO / JPA repositories). In addition to bean registration, @Repository automatically enables Spring's DataAccessException translation mechanism, converting low-level SQL/JPA exceptions into Spring's unified unchecked exception hierarchy.

Q5: How does @Transactional manage database transactions?

ANSWER:

- @Transactional uses Spring AOP (Aspect-Oriented Programming) to dynamically create a proxy around the target class or method.

- When a method annotated with @Transactional (such as UserRepository.updateUserOnlineStatus) is called:

1. The AOP proxy opens/begins a database transaction context before method invocation.

2. If the method completes successfully without throwing an unchecked exception, the proxy commits the transaction to MySQL.

3. If an unchecked exception (RuntimeException or Error) occurs, the proxy automatically triggers a transaction rollback, reverting all database changes made during the method execution.

PART 3: SQL / DBMS QUESTIONS & ANSWERS

Q6: What is the difference between WHERE and HAVING in SQL?

ANSWER:

1. WHERE clause: Filters individual rows BEFORE group aggregation (GROUP BY) takes place. It cannot contain aggregate functions like SUM(), COUNT(), or AVG().
2. HAVING clause: Filters aggregated groups AFTER the GROUP BY operation has been performed. It is specifically designed to evaluate conditions containing aggregate functions (e.g., HAVING COUNT(cm.id) > 10).

Q7: Explain Inner Join vs Left Join with respect to messages and users.

ANSWER:

1. INNER JOIN: Returns only rows where there is a matching value in both tables.
 - Example: `SELECT cm.*, u.email FROM chat_messages cm INNER JOIN users u ON cm.sender = u.username;`
 - Output: Only messages whose sender matches an existing registered user in the users table.
2. LEFT JOIN (Left Outer Join): Returns all rows from the left table (chat_messages), and the matching records from the right table (users).
 - If there is no match in the right table, NULL values are returned for the right table's columns.
 - Output: Returns ALL chat messages even if the sender's user record was deleted or non-existent.

Q8: What indexes would you add to optimize message retrieval?

ANSWER:

To eliminate full table scans on chat_messages, add the following indexes:

1. For Private Message Queries (findPrivateMessagesBetweenTwoUsers):
 - `CREATE INDEX idx_private_msg_1 ON chat_messages(sender, receiver, time_stamp);`
 - `CREATE INDEX idx_private_msg_2 ON chat_messages(receiver, sender, time_stamp);`
 - Reason: Allows MySQL to perform an index range scan for private conversation lookups sorted by timestamp.
2. For Public / Recent Message Queries (findTop50ByMessageTypeOrderByTimeStampDesc):
 - `CREATE INDEX idx_public_msg ON chat_messages(message_type, time_stamp DESC);`
 - Reason: Enables fast filtering by message_type ('CHAT') and direct retrieval of the top 50 rows without incurring a filesort operation.

PART 4: DATA STRUCTURES & ALGORITHMS (DSA)

Q9: How would you implement a Rate Limiter for sending messages? (Token Bucket / Sliding Window)

ANSWER:

1. Token Bucket Algorithm:
 - Maintain a bucket for each user containing up to N tokens (e.g., 10 tokens), refilled at a constant rate (e.g., 2 tokens/sec).
 - Every time a user sends a message over STOMP (/chat.sendMessage), check if a token is available. If yes, decrement 1 token and allow transmission. If 0 tokens remain, drop the message and send a 429 Too Many Requests error frame.

2. Sliding Window Counter (Distributed with Redis):

- Use a Redis Sorted Set (ZSET) keyed by user ID (rate_limit:user:123).
- Store message sent timestamps as scores. Remove timestamps older than the window threshold (e.g., current_timestamp - 60s).
- If ZCARD (element count) in the current window exceeds limit (e.g., 20 messages/min), reject the message.

Q10: What data structure is best suited for storing recent chat messages in memory? (Circular Queue / Deque)

ANSWER:

- Deque (Double-Ended Queue) / ArrayDeque / Circular Buffer.
- Reason:
 1. O(1) time complexity for inserting new messages at the tail (push/offer).
 2. O(1) time complexity for removing oldest messages from the head (poll/pop) when the queue reaches max capacity (e.g., storing the latest 50 messages).
 3. Memory efficient compared to LinkedList since contiguous array memory avoids node pointer overhead.

PART 5: SYSTEM DESIGN & HIGH CONCURRENCY

Q11: How would you scale this app to 100,000 concurrent WebSocket connections? (Netty, AWS ALB, Redis PubSub)

ANSWER:

1. Architecture Shift:
 - Replace standard blocking I/O servlet containers with Netty or Spring WebFlux for non-blocking asynchronous event-loop I/O, reducing thread-per-connection memory footprint.
2. Load Balancing:
 - Deploy multiple stateless Spring Boot backend instances behind an AWS Application Load Balancer (ALB) or NGINX configured with WebSockets support & Sticky Sessions (Cookie/IP hash).
3. External Message Broker:
 - In-memory simple broker (enableSimpleBroker) fails across multiple nodes. Replace with a distributed Pub/Sub layer like Redis Pub/Sub or RabbitMQ STOMP Broker Relay.
 - When User A on Node 1 messages User B on Node 2, Node 1 publishes the event to Redis, Node 2 receives it and pushes it down User B's active WebSocket connection.

Q12: How do you handle message delivery guarantees (At-least-once vs Exactly-once)?

ANSWER:

1. At-Least-Once Delivery:
 - Server attaches a unique message ID (UUID) to every STOMP frame.
 - Client must send a STOMP ACK frame back upon receiving and rendering the message. If no ACK is received within timeout, server re-delivers.

- Client maintains a local deduplication cache of processed message IDs to handle duplicates cleanly.

2. Exactly-Once Delivery:

- Requires idempotent processing: Database schema enforces UNIQUE constraint on message_id (UUID).
- Transactional Outbox Pattern: Save message entity and outbound event atomically in MySQL before publishing over WebSocket.

PART 6: HR & LEADERSHIP QUESTIONS

Q13: Tell me about a difficult bug you faced and how you debugged it.

ANSWER:

- Bug Example: Disconnecting WebSocket session was not updating user online status (isOnline remained true in DB after user closed browser).
- Root Cause Discovery:
 - Inspected server console logs and database records after terminating a WebSocket client session. Observed that while "Connected to web socket" was logged on connect, no disconnect logs were emitted.
 - Inspected WebSocketListener.java line 28 and discovered that handleWebSocketDisconnectListener(SessionDisconnectEvent event) was missing the @EventListener annotation!
- Resolution: Added @EventListener annotation to trigger event handling upon SessionDisconnectEvent, ensuring updateUserOnlineStatus(username, false) executes cleanly.

Q14: How do you prioritize security vs developer velocity when building a project?

ANSWER:

- Ideal Balance: Integrate security by design into foundational architecture rather than retrofitting it late in development.
- Core Security First: Implement essential authentication/authorization primitives (BCrypt password hashing, JWT filters, CORS rules, prepared statements for SQL injection prevention) on Day 1.
- Velocity via Automation: Use automated SAST/DAST security scanning tools, SonarQube, and pre-commit hooks so security checks execute automatically without slowing down feature development.